UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 – HIGH PERFORMANCE DATA PROCESSING

SECTION 01

Project 2: Foodpanda App Reviews Sentiment Analysis

LECTURER:

PROF. MADYA. TS. DR. MOHD SHAHIZAN BIN OTHMAN

SUBMISSION DATE: 26 JUNE 2026

GROUP: CODE COOKIES

STUDENT NAME	MATRIC NO
NAJMA SHAKIRAH BINTI SHAHRULZAMAN	A23CS0140
NURUL ASYIKIN BINTI KHAIRUL ANUAR	A23CS0162
HARINI A/P SANGARAN	A23CS0081

Table of Contents

1.0 Introduction

This section explains the background of the project, the project objectives and the scope of the project.

1.1 Background

Food delivery services have become an essential part of daily life in Malaysia, providing consumers with convenient access to meals through mobile applications. Among these services, Foodpanda is one of the country's leading online food delivery platforms, connecting customers with thousands of restaurants and delivery riders across multiple cities. As the platform continues to expand, millions of users share their experiences through reviews on the Google Play Store, providing valuable insights into customer satisfaction regarding overall user experience, including service quality, customer support and application stability.

These publicly available reviews represent an important source of customer feedback that organizations can utilize to understand user sentiment and identify areas requiring improvement. Automated sentiment analysis supported by real-time data processing technologies has become increasingly important for organisations seeking to monitor customer satisfaction and respond promptly to service issues. Therefore, this project develops a real-time sentiment analysis system that analyses Foodpanda Malaysia Google Play reviews using Apache Kafka and Apache Spark Structured Streaming. Approximately 100,000 reviews were collected from the Malaysian Google Play Store using the google-play-scraper Python library. The collected reviews were preprocessed using Natural Language Processing (NLP) techniques, including text cleaning, tokenization, stopword removal, stemming, and lemmatization.

To classify customer opinions, two machine learning models, namely Naive Bayes and LSTM, were trained and evaluated using standard performance metrics. The best-performing model was then integrated into a streaming architecture, where Apache Kafka continuously streams incoming reviews to Apache Spark for real-time sentiment prediction. The classified results are subsequently stored for visualization and analysis, enabling stakeholders to monitor customer sentiment effectively.

1.2 Objectives

The objectives of this project are:

1. To collect approximately 100,000 Malaysian Foodpanda Google Play reviews using the google-play-scraper library.
2. To preprocess the collected review text using Natural Language Processing (NLP) techniques, including text cleaning, tokenization, stopword removal, stemming, and lemmatization.
3. To develop and compare two sentiment classification models, namely Naive Bayes and LSTM, for classifying reviews into positive, neutral, and negative sentiment.
4. To implement a real-time streaming pipeline using Apache Kafka and Apache Spark Structured Streaming for continuous sentiment prediction.
5. To store the predicted sentiment results for visualization and analysis.

1.3 Project Scope

The scope of this project includes the following components:

1. Collecting approximately 100,000 Foodpanda reviews from the Malaysian Google Play Store using the google-play-scraper Python library.
2. Performing data preprocessing using Natural Language Processing (NLP) techniques including lowercasing, noise removal, tokenization, stopword removal, stemming, and lemmatization.
3. Labelling sentiment classes based on review ratings, where ratings of one and two stars represent negative sentiment, three stars represent neutral sentiment, and four to five stars represent positive sentiment.
4. Training and evaluating two machine learning algorithms which are Naive Bayes and LSTM.
5. Deploying the best-performing model within an Apache Kafka and Apache Spark Structured Streaming pipeline to perform real-time sentiment classification.
6. Storing the classified output for visualization and comparing system performance between batch processing and streaming processing.

2.0 Data Acquisition and Preprocessing

The section explains how Foodpanda application review data is collected and preprocessed.

2.1 Data Collection

The dataset used in this project consists of customer reviews collected from the official Foodpanda application available on the Google Play Store. Foodpanda was selected because it is one of the most widely used food delivery platforms in Malaysia, with a large number of active users who continuously provide feedback regarding their service experiences. The reviews were collected using the google-play-scraper Python library, which provides access to publicly available Google Play reviews without requiring the official Google Play API. The scraper was configured to retrieve English-language reviews from the Malaysian Google Play Store by specifying the language parameter as English (lang='en') and the country parameter as Malaysia (country='my'). To obtain a representative dataset for machine learning and streaming experiments, the scraper repeatedly retrieved reviews in batches of 200 records until approximately 100,000 reviews were collected.

The implementation begins by importing the required Python libraries.

```
from google_play_scraper import reviews, Sort
import pandas as pd
import time
import os
```

The purpose of each library is described below :

- **google_play_scraper** - Retrieves reviews from the Google Play Store.
- **pandas** - Stores and manipulates the collected review data in tabular format.
- **time** - Introduces delays between requests to prevent sending continuous requests to the Google Play servers.
- **os** - Creates the output directory automatically if it does not already exist.

Next, a directory named data is created.

```
os.makedirs("data", exist_ok=True)
```

The parameter exist_ok=True ensures that no error occurs if the folder already exists. This directory is used to store the raw dataset generated by the scraper.

Then, the desired number of reviews are specified.

```
APP_ID = "com.global.foodpanda.android"
TARGET_REVIEWS = 100000
```

The variable APP_ID uniquely identifies the Foodpanda application on the Google Play Store, while TARGET_REVIEWS specifies the maximum number of reviews that the scraper attempts to collect.

Two variables are created for the scraping process.

```
all_reviews = []
continuation_token = None
```

The variable all_reviews stores every review collected throughout the execution of the program. The variable continuation_token is initially assigned None. This token is automatically generated by the Google Play Store after each request and is used to retrieve the next batch of reviews. It enables pagination, allowing the scraper to continue collecting reviews until the target number has been reached.

The review collection process is then performed using a while loop.

```
while len(all_reviews) < TARGET_REVIEWS:
```

The loop continues executing until either 100,000 reviews have been collected, or no additional reviews are available from the Google Play Store. Within each iteration, the reviews() function is executed.

```
result, continuation_token = reviews(
    APP_ID,
    lang="en",
    country="my",
    sort=Sort.NEWEST,
    count=200,
    continuation_token=continuation_token
)
```

APP_ID specifies the Foodpanda application. lang = "en" retrieves English language reviews only whereas country = "my" retrieves reviews from Malaysia Google Play Store only. sort=Sort.NEWEST prioritizes collecting the newest reviews first whereas count = 200

ensures 200 reviews are retrieved per API call. Lastly, continuation_token will proceed the next batch of reviews retrieval from where it ended.

After each request, the scraper checks whether any reviews were returned. If no reviews are returned, the loop terminates because there are no additional reviews available for collection. The code for this function is attached below.

```
if not result:
    print("No more reviews available.")
    break
```

Once all the reviews are successfully retrieved, they are appended to the master list. The total number of collected reviews is then displayed from time to time for every API call to indicate that the code is running correctly.

```
all_reviews.extend(result)

print(f"Collected: {len(all_reviews)} reviews")
```

Example output:

Collected: 200 reviews
Collected: 400 reviews
Collected: 600 reviews
…
Collected: 100000 reviews

Google Play provides a continuation token only when more reviews are available. If no continuation token is returned, the scraper terminates because all accessible reviews have already been collected. The code for this function is attached below.

```
if not result:
    print("No more reviews available.")
    break
```

A one-second delay is then inserted after each iteration. This delay reduces the frequency of requests sent to the Google Play Store, improving stability and reducing the likelihood of temporary connection issues.

```
time.sleep(1)
```

After all reviews have been collected, the data is converted into a Pandas DataFrame. Instead of storing the complete review object, only the relevant attributes required for sentiment analysis are extracted.

```
df = pd.DataFrame([{
    "review_id": r.get("reviewId"),
    "review_text": r.get("content"),
    "rating": r.get("score"),
    "review_date": r.get("at"),
    "thumbs_up": r.get("thumbsUpCount"),
    "app_version": r.get("reviewCreatedVersion"),
    "country": "Malaysia",
    "source": "Google Play",
    "app_name": "Foodpanda"
} for r in all_reviews])
```

The collected attributes are summarised in table below :

Field	Description
review_id	Unique identifier assigned to each review
review_text	Review text written by the user
rating	User rating ranging from 1 to 5 stars
review_date	Date and time when the review was posted
thumbs_up	Number of users who marked the review as helpful
app_version	Version of the Foodpanda application used by the reviewer
country	Country of the Google Play Store (Malaysia)
source	Source platform (Google Play Store)

app_name	Application name (Foodpanda)

Then, duplicate reviews are removed based on the review ID and reviews without textual content are discarded. This ensures that only valid review text is retained for the preprocessing stage. The python code is attached below.

```
df = df.drop_duplicates(subset=["review_id"])
df = df[df["review_text"].notna()]
```

The final dataset is then exported as a CSV file.

```
df.to_csv("data/foodpanda_malaysia_reviews_raw.csv", index=False, encoding="utf-8-sig")
```

Finally, the scraper displays the total number of collected reviews.

```
print("Finished scraping.")
print("Total reviews saved:", len(df))
```

Example:

Finished scraping.
Total reviews saved: 100000

The entire compiled code is attached in Appendix B.

2.2 Data Preprocessing

The raw Foodpanda review dataset was preprocessed using several Natural Language Processing (NLP) techniques to improve the quality and consistency of the text before training the sentiment classification models. The preprocessing process involved loading the dataset, cleaning the review text, tokenizing the words, removing stopwords, applying stemming and lemmatization, assigning sentiment labels, and exporting the processed dataset.

The preprocessing script begins by importing the required Python libraries and loading the raw review dataset collected during the data acquisition stage and the required NLTK resources are downloaded before preprocessing.

```
import pandas as pd
import re
import nltk
import os

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

os.makedirs("data", exist_ok=True)

nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")

df = pd.read_csv("data/foodpanda_malaysia_reviews_raw.csv")
```

The code below ensures reviews without textual content are removed to ensure that only valid reviews are processed.

```
TEXT_COLUMN = "review_text"
df = df[df[TEXT_COLUMN].notna()]
```

Next, before preprocessing begins, the stopword list, stemmer, and lemmatizer are initialized.

```
stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
```

These components perform different NLP tasks :

- **Stopwords** - Removes common English words (e.g., the, is, and)
- **Porter Stemmer** - Reduces words to their root form
- **WordNet Lemmatizer** - Converts words into their dictionary base form

Then, the review text is standardized by converting all characters to lowercase, removing URLs, punctuation, numbers, special characters, and extra whitespace. This step standardizes the review text and removes unnecessary information that may negatively affect model performance.

```python
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", " ", text)
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

After cleaning, each review is split into individual words (tokens). Common English stopwords are removed, followed by stemming and lemmatization to reduce vocabulary size while preserving semantic meaning.

```python
def preprocess_text(text):
    cleaned = clean_text(text)
    tokens = cleaned.split()

    tokens = [
        word for word in tokens
        if word not in stop_words and len(word) > 2
    ]

    stemmed_tokens = [stemmer.stem(word) for word in tokens]
    lemmatized_tokens = [lemmatizer.lemmatize(word) for word in tokens]

    return pd.Series({
        "cleaned_text": cleaned,
        "tokens": " ".join(tokens),
        "stemmed_text": " ".join(stemmed_tokens),
        "lemmatized_text": " ".join(lemmatized_tokens)
    })
```

The preprocessing function is then applied to every review in the dataset.

```python
processed = df[TEXT_COLUMN].apply(preprocess_text)
df = pd.concat([df, processed], axis=1)
```

An example of the transformation is shown below :

Stage	Output
Original	The delivery was very fast!
Cleaned	the delivery was very fast

Tokens	["the", "delivery", "was", "very", "fast"]
Stopwords Removed	["delivery", "fast"]
Stemmed	deliver fast
Lemmatized	delivery fast

Since Google Play reviews do not provide sentiment labels, the review ratings are used to generate the target classes for supervised learning. As per the code, reviews with 1 or 2 stars are labelled as negative, reviews with 3 stars are labelled as neutral and reviews with 4 or 5 stars are labelled as positive.

```
def label_sentiment(rating):
    rating = int(rating)
    if rating <= 2:
        return "negative"
    elif rating == 3:
        return "neutral"
    else:
        return "positive"
```

To improve data quality, duplicate reviews are removed based on the lemmatized review text. Reviews with insufficient textual information are also excluded.

```
df = df.drop_duplicates(subset=["lemmatized_text"])
df = df[df["lemmatized_text"].str.len() > 5]
```

Finally, the processed dataset is exported as a CSV file.

```
df.to_csv("data/foodpanda_reviews_preprocessed.csv", index=False, encoding="utf-8-sig")
```

The resulting dataset, foodpanda_reviews_preprocessed.csv, contains the original review information together with the cleaned text, tokenized words, stemmed text, lemmatized text, and sentiment labels. This dataset serves as the input for the subsequent sentiment model development stage.

The entire compiled code is attached in Appendix C.

2.3 Cleaning Process

The tools and technologies used throughout the development of the proposed sentiment analysis pipeline are summarized in the table below :

Category	Tools Used	Description
Data Collection	Google Play Scraper	Scrapes Foodpanda user reviews from the Malaysian Google Play Store.
Data Preprocessing	Python, Pandas, NLTK	Cleans and preprocesses review text through lowercasing, noise removal, tokenization, stopword removal, stemming, and lemmatization.
Feature Extraction	TF-IDF Vectorizer	Converts textual reviews into numerical feature vectors for model training.
Machine Learning Model	Naive Bayes	Trains a traditional machine learning model for sentiment classification.
Deep Learning Model	LSTM	Develops and trains a Long Short-Term Memory (LSTM) model for sentiment prediction.
Streaming & Processing	Apache Kafka, Apache Spark Structured Streaming (PySpark)	Streams review data in real time and performs continuous sentiment prediction.
Data Storage	Elasticsearch	Stores and indexes predicted sentiment results for efficient retrieval.

Data Visualization	Kibana	Visualizes sentiment distributions, trends, and analytics through interactive dashboards.
Deployment	Docker	Containerizes and manages the services used in the streaming pipeline.
Development Environment	Python, Google Colab, Jupyter Notebook	Used for data preprocessing, model development, experimentation, and testing.

3.0 Sentiment Model Development

This section provides an introduction to the development of the sentiment model in this project. The foodpanda reviews were preprocessed, and then the two different models were built to train and test the data.

3.1 Model Choice

In this project, two machine learning models were chosen, which are Naive Bayes and Long Short-Term Memory (LSTM). The Naive Bayes is selected as it is widely used for text classification, and is easier to train. Additionally, LSTM was chosen because of its ability to learn sequential patterns and context information from text, which enables it to better understand the relationship between words in a sentence.

The same preprocessed foodpanda review dataset was used for both training and testing of both models so that the two models could be fairly compared. They were evaluated based on the different evaluation metrics, and the one that has the best result was chosen for deployment in the sentiment analysis system. The comparison between two models is given in Section 6.2 Model Comparison.

3.2 Training Process

The training model process was implemented in Google Colab using Python, Scikit-learn, and Tensorflow libraries. The training workflow includes dataset preparation, label encoding, feature extraction, model training, and model evaluation.

3.2.1 Dataset Preparation

The model training process started with loading the preprocessed FoodPanda reviews dataset which is saved in the file foodpanda_reviews_preprocessed.csv. The features chosen for sentiment classification were only those that were pertinent to the task, namely lemmatized_text, rating, review_date, thumbs_up, and sentiment. To ensure that there was valid text for model training, reviews with missing values in the lemmatized_text column were dropped. The Python code, which loaded the dataset and extracted the columns and dropped the records which have missing review text before training is shows in Figure 3.1.

```
df = df[['lemmatized_text', 'rating', 'review_date', 'thumbs_up', 'sentiment']]

df = df.dropna(subset=['lemmatized_text'])
```

Figure 3.1: Preprocessed foodpanda Review Dataset

3.2.2 Label Encoding

In this project, the sentiment classes used in the dataset were preprocessed. During the preprocessing stage, reviews with ratings of 1–2 were labelled as negative, a rating of 3 as neutral, and ratings of 4–5 as positive. Hence, there was no need to label sentiments when developing the model.

Machine learning models need numerical values, not text labels. So the LabelEncoder from scikit-learn was used to map the labels of sentiment into numbers. The encoded labels were then used for the target variable in model training. The mapping between the sentiment labels and their encoded values is presented in Table 3.1.

Table 3.1: Sentiment Label Encoding

Sentiment	Encoded Label
Negative	0
Nutral	1
Postive	2

3.2.3 Data Splitting

The sentiment labels were then encoded and the data split into training and testing dataset with the train_test_split() function. 80 % of the data set was used for training the model, and 20 % was used for testing the model. Stratified sampling was used to maintain the distribution of sentiment classes in both datasets and a random seed of 42 was provided to guarantee reproducibility. The code to split the data into train and test sets is illustrated in Figure 3.2.

```
X = df['lemmatized_text']
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Figure 3.2: Splitting the dataset into Training and Testing Sets

3.2.4 Feature Extraction

Different feature extraction techniques were applied for the two classification models. The review text was transformed into numerical feature vectors for the Naive Bayes model using the TF-IDF (Term Frequency-Inverse Document Frequency) approach. TF-IDF is a method for determining the importance of words in a document while reducing the impact of words that appear frequently. In order to balance computing speed with vocabulary word retention, the maximum vocabulary size was set at 5,000. Figure 3.3 illustrates the application of the TF-IDF feature extraction process to the Multinomial Naive Bayes model

```
tfidf = TfidfVectorizer(max_features=5000)

X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

Figure 3.3: Generating TF-IDF Feature Vectors for the Multinomial Naive Bayes Model

The LSTM model required that the review's text be tokenized with the Keras Tokenizer before being translated to numerical sequences. These sequences were then expanded to a set length to ensure that all reviews were of identical length before being fed into the neural network. Figure 3.4 illustrates the tokenization and sequence padding procedure to the LSTM model.

```
max_words = 10000
max_len = 100

tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

X_train_pad = pad_sequences(X_train_seq, maxlen=max_len)
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len)
```

Figure 3.4: Tokenizing and Padding Text Sequences for the LSTM Model

3.2.5 Model Training and Evaluation

The same training and testing datasets were used to train and assess the Multinomial Naive Bayes and LSTM models following the feature extraction procedure. Table 3.2 provides a summary of the entire model creation process.

Table 3.2: Training and Evaluation Process of the Sentiment Classification Model

Step	Description
Model Initialization	Multinomial Naive Bayes used TF-IDF features, while LSTM used an Embedding layer, LSTM layer, Dropout layer, and Dense output layer.
Model Training	Multinomial Naive Bayes was trained on TF-IDF vectors, while LSTM was trained on tokenized and padded sequences.
Prediction	Both models generated sentiment predictions using the testing dataset after the training process was completed.
Model Evaluation	Performance was measured using accuracy, precision, recall, F1-score, and confusion matrix.
Model Comparison	Both models were compared using accuracy and weighted F1-score.
Model Saving	The trained models and supporting files were saved for deployment.

The implementation of the Multinomial Naive Bayes model is shown in Figure 3.5. This model was trained using TF-IDF feature vectors generated from the preprocessed review text.

```
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
```

```
MultinomialNB
MultinomialNB()
```

```
y_pred_nb = nb_model.predict(X_test_tfidf)

print("NAIVE BAYES RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_nb))
print(classification_report(y_test, y_pred_nb,target_names=le.classes_))
```

Figure 3.5: Training the Multinomial Naive Bayes Model

Figure 3.6 presents the implementation of the LSTM model. The model architecture consists of an embedding layer, an LSTM layer, a dropout layer, and a dense output layer.

```
model = tf.keras.Sequential([

    tf.keras.layers.Embedding(
        input_dim=max_words,
        output_dim=128,
        input_length=max_len
    ),

    tf.keras.layers.LSTM(64),

    tf.keras.layers.Dropout(0.3),

    tf.keras.layers.Dense(
        num_classes,
        activation='softmax'
    )

])
```

Figure 3.6: Building the LSTM Model

The testing dataset was then used to evaluate both models using the metrics listed in Table 3.3.

Table 3.3: Evaluation Metrics Used for Model Performance Assessment

Metric	Description
Accuracy	Shows how many reviews were classified correctly out of all the reviews in the test set.
Precision	Shows how many of the reviews predicted for each sentiment class were actually correct.
Recall	Measures the ability of the model to correctly identify actual instances for each sentiment class.
Weighted F1-score	Gives a balanced score based on precision and recall, while also taking into account the number of reviews in each class.
Confusion Matrix	Shows the number of correct and incorrect predictions made for each sentiment class.

Figure 3.7 shows the code used to compare the performance of both models. The evaluation results were measured using accuracy and weighted F1-score to determine which model performed better.

```
print("===== MODEL COMPARISON =====")

print("Naive Bayes Accuracy :", round(nb_acc, 4))
print("LSTM Accuracy        :", round(lstm_acc, 4))

print()

print("Naive Bayes F1 Score :", round(nb_f1, 4))
print("LSTM F1 Score        :", round(lstm_f1, 4))
```

Figure 3.7: Comparing the Performance of the Multinomial Naive Bayes and LSTM Models

3.3 Evaluation

The performance of the Multinomial Naive Bayes and LSTM models was evaluated using the testing dataset. The evaluation focused on two performance metrics, which are accuracy and weighted F1 score. Table 3.4 shows the comparison results for both models.

Table 3.4: Performance Comparison of the Sentiment Classification Models

Model	Accuracy	Weighted F1-score
Naive Bayes	84.62%	0.8241
LSTM	87.00%	0.8530

The results from Table 3.4 showed that the LSTM model performed best with an accuracy of 87.00% and a weighted F1 score of 0.8530, exceeding the accuracy and weighted F1 score of the Multinomial Naive Bayes model, which were 84.62% and 0.8241, respectively. The results showed that LSTM model outperformed the rest of the models in FoodPanda sentiment classification.

4.0 Apache System Architecture

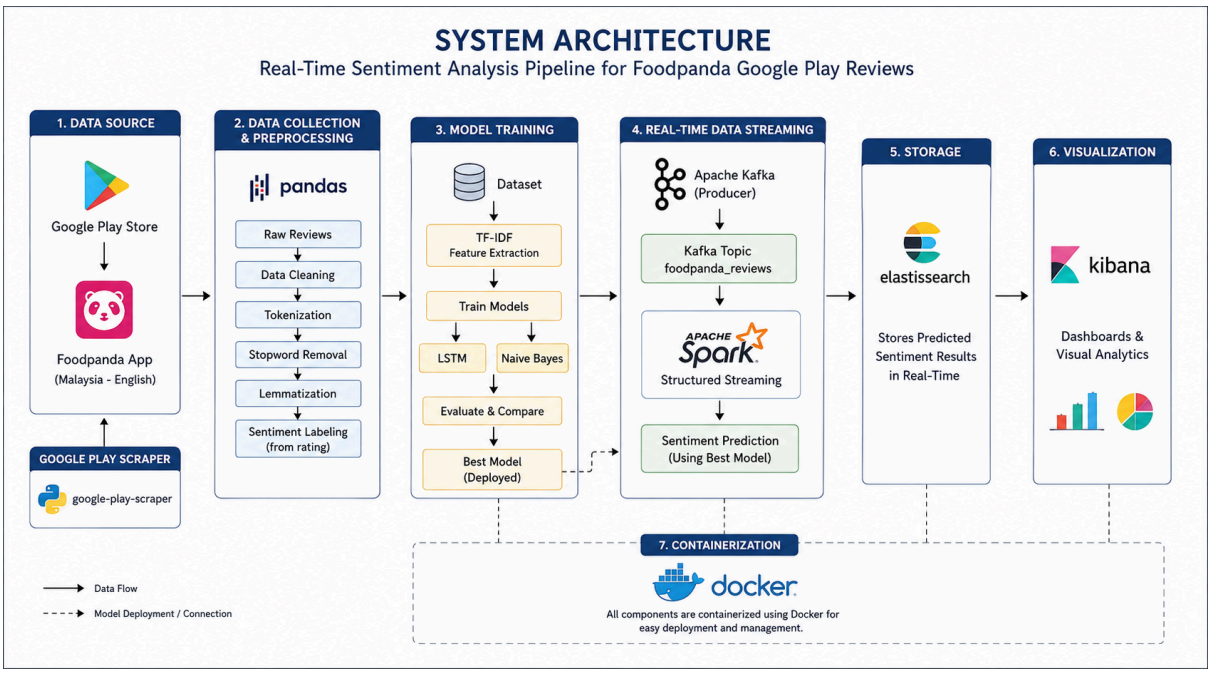


Figure 4.1 FoodPanda Sentiment Analysis System Architecture

Figure 4.1 above illustrates the overall architecture of the proposed real-time sentiment analysis system. The pipeline begins by collecting Foodpanda reviews from the Malaysian Google Play Store using google-play-scraper. The collected reviews are then preprocessed using Python, Pandas, and NLTK, where text cleaning, tokenization, stopword removal, stemming, and lemmatization are performed before sentiment labels are assigned.

Next, the preprocessed reviews are transformed into numerical features using TF-IDF and used to train two sentiment classification models: LSTM and Naive Bayes. After evaluating both models, the best-performing model is selected for deployment. Although the LSTM model achieved higher classification performance than Naive Bayes model during the training and evaluation stage, the Naive Bayes model was selected for deployment in the real-time streaming pipeline. as it has significantly lower computational and memory requirements, enabling more stable execution within the available hardware resources.

For real-time processing, the review data is streamed through Apache Kafka, where Apache Spark Structured Streaming continuously accepts incoming reviews and performs real-time sentiment prediction using the deployed model. The prediction results are stored in

Elasticsearch and visualized through interactive Kibana dashboards. All major services are deployed using Docker, providing a consistent and portable execution environment.

5.0 Analysis and Results

This section explains about all the analysis charts in the FoodPanda Customer Review Sentiment dashboard that was created using ElasticSearch and Kibana after real-time streaming and sentiment classification using Kafka and Spark. The full dashboard is shown in Appendix A. The purpose of this dashboard is to understand customer review patterns, review rating distribution and sentiment trends.

5.1 Dashboard Overview

Figure 5.1 shows the total reviews from the FoodPanda App which is 16,962 reviews after the real-time streaming is conducted. The metric card displays the total reviews by counting the number of records in the Kibana data view.

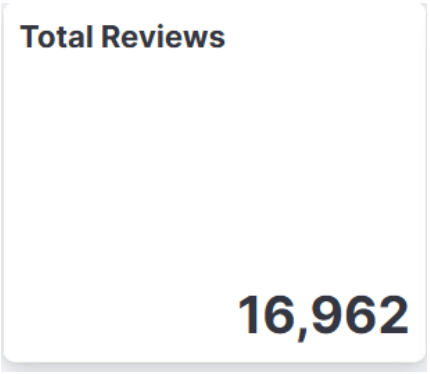


Figure 5.1 Total Reviews Metric Card

Figure 5.2 shows the average star rating of the FoodPanda App reviews. It shows a value of 2.445 out of 5 which means that the overall customer rating is low. Since the value is also below the midpoint of 3, this suggests that FoodPanda customers' feedback lean towards dissatisfaction. This finding is supported by the sentiment distribution displayed in Figure 5.3 where negative reviews make up the majority of the reviews.

Figure 5.2 Average Star Rating Metric Card

Figure 5.3 displays a bar chart that shows the review distribution by star rating. The chart shows that the highest total reviews recorded have 1 star rating indicating that most customers gave very low ratings and are dissatisfied with the app and service. Then it is followed by 5-star reviews which shows that some customers still had positive experiences. However, 2-star, 3-star and 4-star ratings have lower total reviews with 3 stars being the lowest at 516 total reviews. The patterns shows that the customer reviews are strongly divided between very negative and very positive experience, but negative reviews are more dominant across all reviews.

Figure 5.3 Review Distribution by Star Rating

Figure 5.4 is a pie chart that illustrates the sentiment distribution of all recorded FoodPanda reviews. The pie chart shows that 68.96% of all reviews are negative while 31.04% are positive reviews. The result suggests that many customers expressed dissatisfaction in their review.

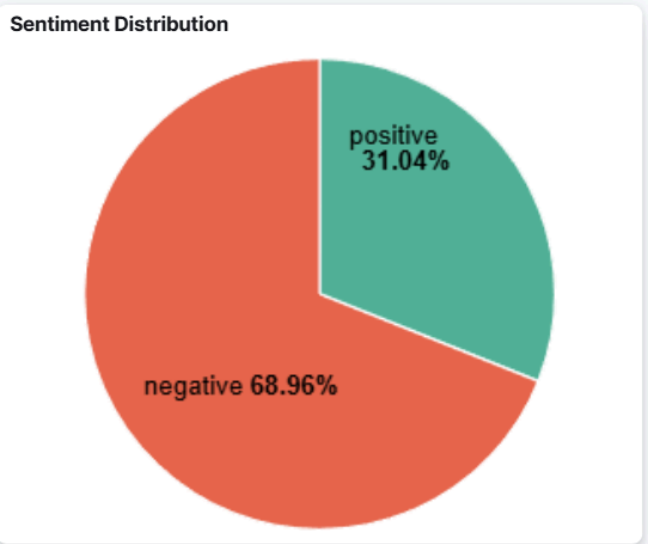


Figure 5.4 Sentiment Distribution Across All FoodPanda Reviews

Figure 5.5 is a line chart that illustrates the daily review trend by both negative and positive sentiment. It displays the recorded daily total reviews in the past year. Based on the trend, negative reviews are consistently higher than positive reviews throughout the one year period. There are noticeable spikes in negative reviews around April to June of 2026 which may indicate periods where customers experienced more service-related issues. The positive review remains lower and stable compared to the negative review trend.

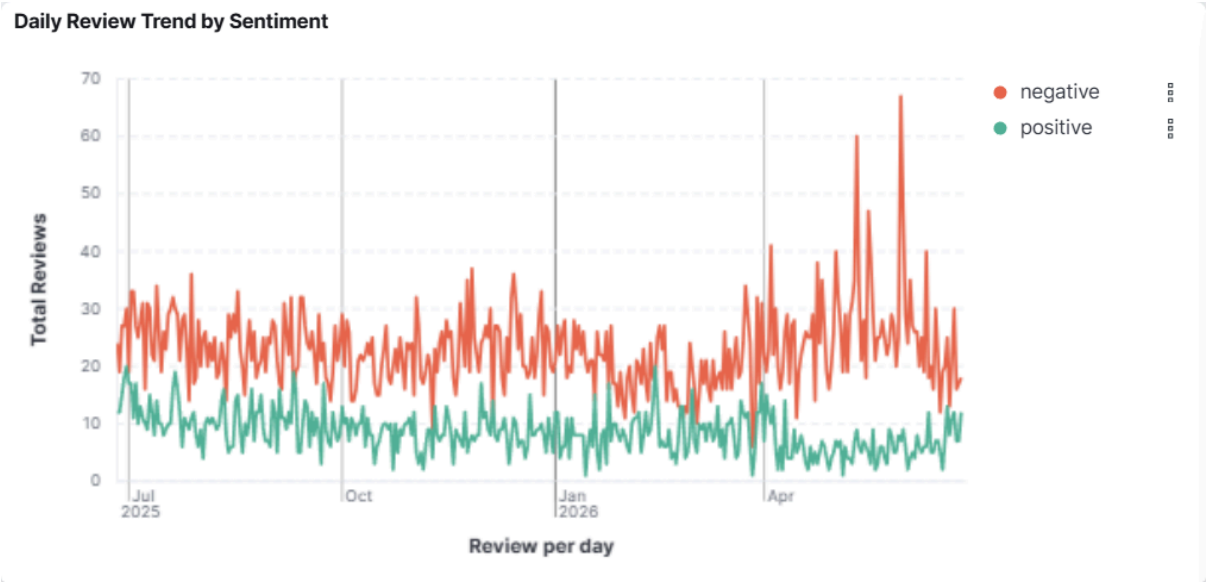


Figure 5.5 Daily Review Trend by Sentiment

Figure 5.6 shows a tag cloud that displays the top 25 most frequent keywords mentioned in the FoodPanda reviews. The largest shown keywords are cancel, payment, refund, time and hour. These keywords suggest that the most common customer issues are related to order cancellation, payment problems, refund requests, and long waiting times. The tag cloud helps in identifying the main complaint topics that appear repeatedly in reviews.

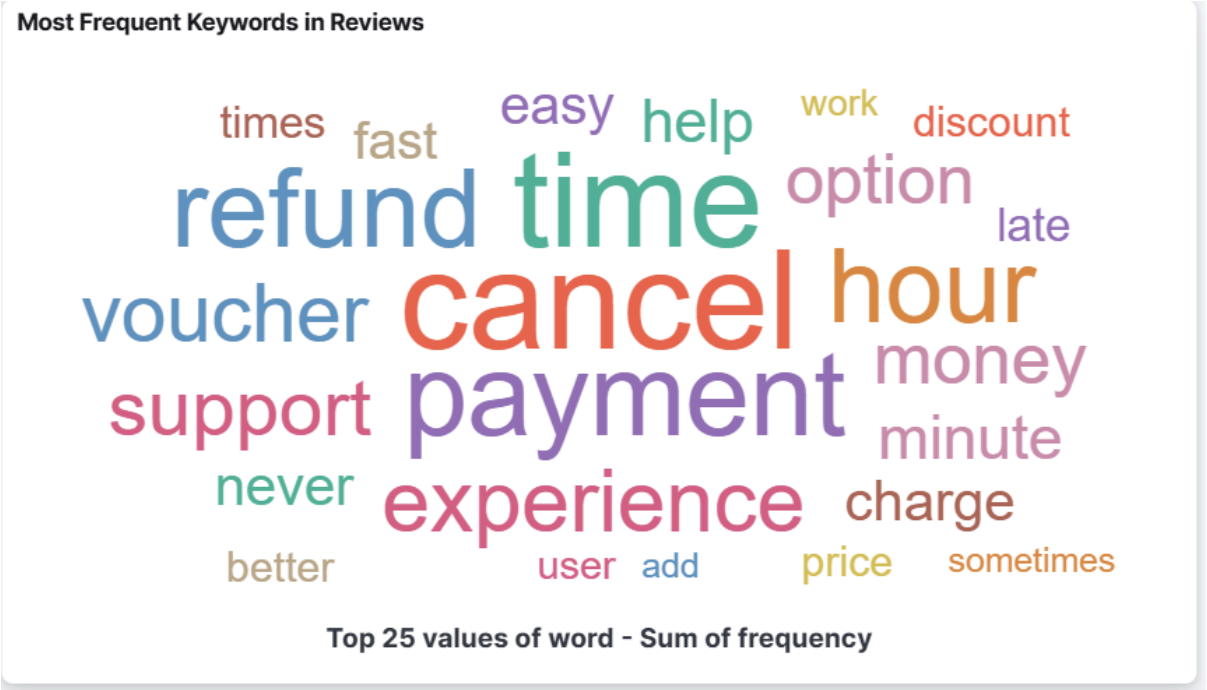


Figure 5.6 Top 25 Most Frequent Keywords in Reviews

5.2 Key Insights and Recommendations

The Foodpanda Customer Review Sentiment dashboard shows that customer dissatisfaction is mainly related to service and operational issues. This can be seen from the low average star rating, a lot of 1-star reviews, and the fact that most of its reviews were negative, which shows that there were issues that many users faced when using the Foodpanda app. Furthermore, the keyword analysis indicates that the primary issues are order cancellation, payment issues, refund issues, and long wait times. The daily sentiment trend also indicates that negative sentiment is always higher than positive sentiment, suggesting that dissatisfaction is not confined to a particular window, but rather is a persistent concern.

From these key insights gained, there are things that Foodpanda needs to do to improve the process of service recovery and customer support. Focus should be on minimizing the

number of orders that are cancelled, processing refunds faster, ensuring the reliability of payment processing, and offering quicker responses via customer support channels. The dashboard will also serve as a monitoring tool that will help the company to identify any sudden surge in negative reviews, and take appropriate action in order to respond swiftly before the situation escalates into something bigger. In conclusion, the analysis results indicate that increasing the operational reliability and addressing complaints can decrease negative sentiment and enhance customer satisfaction.

6.0 Optimization and Comparison

This section discusses the optimizations that were applied during the project and the comparison before and after the optimizations. The comparison focuses on two main parts of the project which are sentiment classification model and the data streaming pipeline architecture.

6.1 Model Comparison

Two models for sentiment classification, Multinomial Naive Bayes and Long Short-Term Memory (LSTM), were built and tested for this project. The same preprocessed FoodPanda review dataset was used to train both the models, while the same testing dataset was used to test both models. Two major evaluation metrics, namely accuracy and weighted F1 score were used for the comparison. The output of the evaluation of the Multinomial Naive Bayes model created in Google Colab is displayed in Figure 6.1.

```
NAIVE BAYES RESULTS
Accuracy: 0.846203242896131
              precision    recall  f1-score   support

    negative       0.83      0.97      0.89      7984
     neutral       0.00      0.00      0.00       522
    positive       0.89      0.71      0.79      3952

    accuracy                           0.85     12458
   macro avg       0.57      0.56      0.56     12458
weighted avg       0.82      0.85      0.82     12458
```

Figure 6.1: Multinomial Naive Bayes Model Evaluation Output

The evaluation output of the LSTM model is shown in Figure 6.2.

```
LSTM RESULTS
Accuracy: 0.8699630759351421
              precision    recall  f1-score   support

    negative       0.89      0.94      0.91      7984
     neutral       0.14      0.01      0.02       522
    positive       0.85      0.84      0.84      3952

    accuracy                           0.87     12458
   macro avg       0.62      0.60      0.59     12458
weighted avg       0.84      0.87      0.85     12458
```

Figure 6.2: LSTM Model Evaluation Output

The overall performance comparison between both models is summarised in Table 6.1.

Table 6.1: Performance Comparison of the Sentiment Classification Models

Model	Accuracy	Weighted F1-score
Naive Bayes	84.62%	0.8241

LSTM	87.00%	0.8530

Based on Table 6.1, the LSTM model achieved better performance than the Multinomial Naive Bayes model, obtaining an accuracy of 87.00% and a weighted F1-score of 0.8530, compared to 84.62% and 0.8241, respectively. Although both models performed well in classifying negative and positive reviews, both showed lower performance for the neutral class due to the smaller number of neutral reviews in the dataset.

6.2 Pipeline Architecture Optimization

The pipeline for sentiment analysis uses Kafka, Spark, ElasticSearch and Kibana. During the development of this project, several issues were encountered and optimizations were applied to resolve these issues.

6.2.1 Kafka Producer Optimization

The first problem that arose is that the Kafka producer could not connect to Kafka broker. This is because the Kafka producer was configured to send the reviews to localhost:9092, and the Kafka broker was not running or was not configured. So, the producer responded with a NoBrokersAvailable error.

The problem was resolved by making sure that Kafka was started correctly before the producer. The Kafka storage was also formatted as Kafka 4.x requires KRaft storage configuration to run the server. Once Kafka was successfully started, the new topic foodpanda_reviews was created. This enabled the producer to push review messages out on a continuous basis to the Kafka topic.

Table 6.2 Kafka Producer Optimization Comparison

Problem	**Cause**	**Optimization**
NoBrokersAvailable error	Kafka broker was not running on localhost:9092	Start Kafka before running the producer
Kafka server failed to start	Kafka storage was not formatted	Format Kafka KRaft storage before starting the server
Topic not found	Kafka topic had not been created	Create the foodpanda_reviews topic

		before streaming
Duplicate or repeated testing issues	Producer was run multiple times during testing	Limit the number of reviews sent during testing to control the data flow

These optimisations ensured that the Kafka producer can be run smoothly and could stream Foodpanda reviews into the Kafka topic successfully.

6.2.2 Spark Streaming Consumer Optimization

Spark component is used to read incoming real-time reviews from Kafka. The Spark streaming subscribes to foodpanda_reviews topic and reads the review data as a stream. An optimization that was applied to the Spark script is the use of forEachBatch. This enables the streaming data to be processed in small batches with the trained machine-learning model. This will enhance processing efficiency for continuous review streaming.

Another optimization was the implementation of checkpointing using checkpointLocation. Checkpointing will allow the streaming job to recover from failure and resume its processing from the last checkpoint rather than from the start.

Table 6.3 Spark Streaming Optimizations Comparison

Problem	**Cause**	**Optimization**
Kafka server failed to start	Model prediction was originally more suitable for batch data	Use foreachBatch to apply the model on each micro-batch
Risk of stream failure or restart	Streaming applications may stop due to errors	Use checkpointing to allow recovery from the last processed state

This optimization helped Spark to process incoming Foodpanda reviews from real-time Kafka messages more reliably and prepare the sentiment results for dashboard visualisation.

6.2.3 Elasticsearch and Kibana Optimization

The processed sentiment results were stored and indexed into Elasticsearch and the dashboard visualisation was done using Kibana. Connectivity problems were observed when trying to connect to Elasticsearch with security authentication enabled during development. This led to

the Python client generating an authentication error since it was trying to connect without any username and password.

Initially, CSV files were used to create dashboards in Kibana, but this was not suitable for the real-time nature of the pipeline that needed continuous update. The data obtained from streaming is stored in an index called foodpanda_streaming_reviews. A data view foodpanda_reviews was created and can then be used for creating charts.

Table 6.4 Elasticsearch and Kibana Optimizations Comparison

Problem	**Cause**	**Optimization**
Python could not connect to Elasticsearch	Elasticsearch security required authentication	Disable security for local development or provide username and password
Dashboard needed updated results	Static CSV upload did not support continuous updates	Store streaming prediction results in Elasticsearch for Kibana visualisation

6.2.4 Docker Deployment Optimization

Components in the pipeline such as Elasticsearch, Kibana and Kafka were isolated in containers to simplify setup using Docker. This reduced dependency since each of the services could be installed on its own with a different environment required. This also prevents version conflicts between local Java, Kafka, Elasticsearch and Kibana installations.

Table 6.5 Docker Optimizations Comparison

Aspect	**Before Docker**	**After Docker**
Service setup	Manual installation and configuration	Services run in isolated containers
Version management	Possible conflicts with local software versions	Fixed container image versions can be used
Port configuration	Manual checking and configuration required	Ports are mapped clearly, such as 9200 for Elasticsearch and 9092 for Kafka

Deployment consistency	May behave differently on different machines	Same container setup can be reused
Troubleshooting	Harder to identify environment-related issues	Easier to restart, remove, or recreate containers

Docker provided benefits to the pipeline and made deployment more consistent, portable, and easy to manage. It helped to eliminate configuration issues and help the project parts run in a more controlled environment. This optimisation enables a seamless flow of information between Kafka, Spark Streaming, Elasticsearch and Kibana, particularly for real-time sentiment analysis and dashboard visualization.

7.0 Conclusion & Future Work

This project managed to create a real-time sentiment analysis system for the FoodPanda customer reviews. The system could gather review information, preprocess the text, classify customer sentiment, and create a visualisation based on the output using a Kibana dashboard. Two popular sentiment classification models, Multinomial Naive Bayes and Long Short-Term Memory (LSTM), were built and tested on the same preprocessed data set. The results of the evaluation showed that the LSTM model had the best performance with an accuracy of 87.00% and a weighted F1 score of 0.8530. The dashboard also gave useful insights about customers' feedback by showing the distribution of customer sentiments, customers' ratings trends, reviews trends, and top mentioned keywords. These visualisations could support FoodPanda to track customer satisfaction and the common problems to be fixed.

Even though the project's goals were met, there are still a number of aspects that might be strengthened in future projects. First, the dataset can be increased by collecting more current reviews and increasing the amount of neutral reviews, which may help the model perform better in the neutral sentiment class. Second, more complex deep learning models can be tested to see if they can outperform the LSTM model in terms of sentiment categorization accuracy. Finally, the dashboard can be improved by adding new visualisations, such as sentiment trends by month. These enhancements may provide more specific information and make the system more helpful for tracking client feedback.

Overall, this project demonstrates that combining machine learning with real-time data streaming and dashboard visualisation can provide valuable insights into customer opinions and support better decision-making for improving FoodPanda's services.

References

Apache Kafka. (n.d.). Apache Kafka documentation. Apache Software Foundation. Retrieved June 26, 2026, from https://kafka.apache.org/documentation/

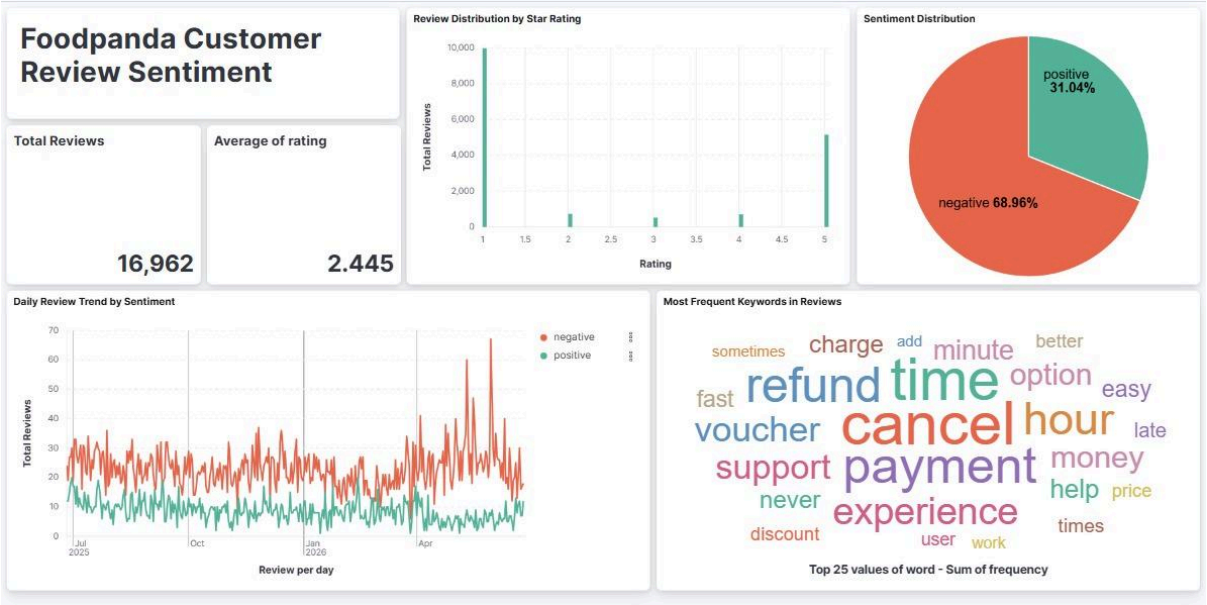
Apache Spark. (n.d.). Structured Streaming programming guide. Apache Software Foundation. Retrieved June 26, 2026, from https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

Docker. (n.d.). What is Docker? Docker Docs. Retrieved June 26, 2026, from https://docs.docker.com/get-started/docker-overview/

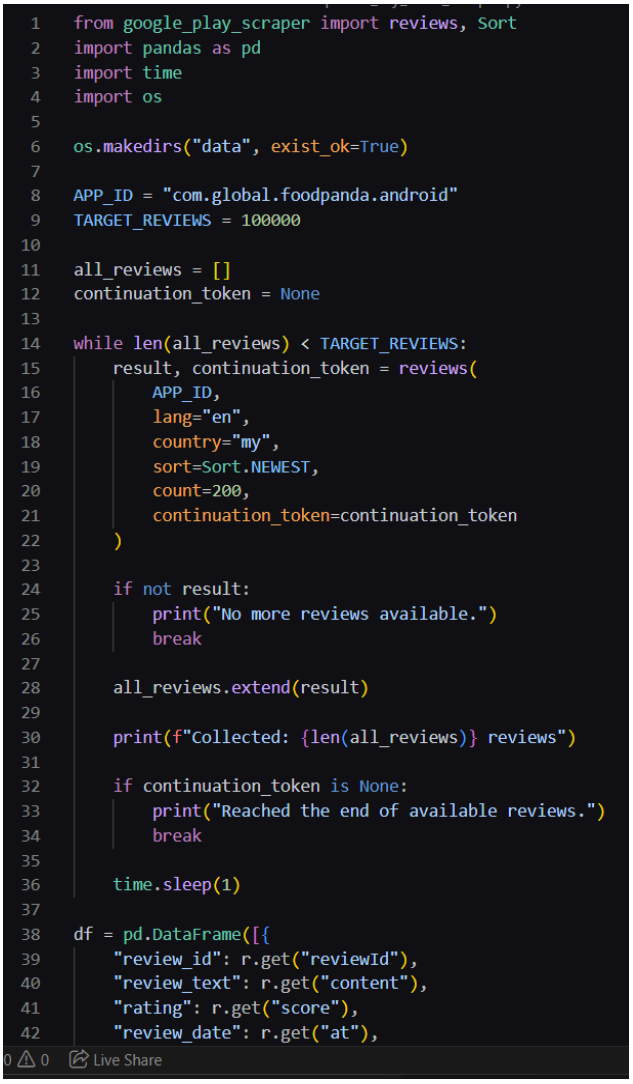
Elastic. (n.d.). Dashboards. Elastic Docs. Retrieved June 26, 2026, from https://www.elastic.co/docs/explore-analyze/dashboards

Google Play Scraper. (n.d.). google-play-scraper. PyPI. Retrieved June 26, 2026, from https://pypi.org/project/google-play-scraper/

Appendix



Appendix A: Foodpanda Customer Review Sentiment Dashboard



```python
from google_play_scraper import reviews, Sort
import pandas as pd
import time
import os

os.makedirs("data", exist_ok=True)

APP_ID = "com.global.foodpanda.android"
TARGET_REVIEWS = 100000

all_reviews = []
continuation_token = None

while len(all_reviews) < TARGET_REVIEWS:
    result, continuation_token = reviews(
        APP_ID,
        lang="en",
        country="my",
        sort=Sort.NEWEST,
        count=200,
        continuation_token=continuation_token
    )

    if not result:
        print("No more reviews available.")
        break

    all_reviews.extend(result)

    print(f"Collected: {len(all_reviews)} reviews")

    if continuation_token is None:
        print("Reached the end of available reviews.")
        break

    time.sleep(1)

df = pd.DataFrame([{
    "review_id": r.get("reviewId"),
    "review_text": r.get("content"),
    "rating": r.get("score"),
    "review_date": r.get("at"),
```

```
    "thumbs_up": r.get("thumbsUpCount"),
    "app_version": r.get("reviewCreatedVersion"),
    "country": "Malaysia",
    "source": "Google Play",
    "app_name": "Foodpanda"
} for r in all_reviews])

df = df.drop_duplicates(subset=["review_id"])
df = df[df["review_text"].notna()]

df.to_csv("data/foodpanda_malaysia_reviews_raw.csv", index=False, encoding="utf-8-sig")

print("Finished scraping.")
print("Total reviews saved:", len(df))
```

Appendix B: Foodpanda Reviews Data Collection Code

```python
import pandas as pd
import re
import nltk
import os

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

os.makedirs("data", exist_ok=True)

nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")

df = pd.read_csv("data/foodpanda_malaysia_reviews_raw.csv")

TEXT_COLUMN = "review_text"
df = df[df[TEXT_COLUMN].notna()]

stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

def label_sentiment(rating):
    rating = int(rating)
    if rating <= 2:
        return "negative"
    elif rating == 3:
        return "neutral"
    else:
        return "positive"

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", " ", text)
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def preprocess_text(text):
    cleaned = clean_text(text)
    tokens = cleaned.split()
```

```
    tokens = [
        word for word in tokens
        if word not in stop_words and len(word) > 2
    ]

    stemmed_tokens = [stemmer.stem(word) for word in tokens]
    lemmatized_tokens = [lemmatizer.lemmatize(word) for word in tokens]

    return pd.Series({
        "cleaned_text": cleaned,
        "tokens": " ".join(tokens),
        "stemmed_text": " ".join(stemmed_tokens),
        "lemmatized_text": " ".join(lemmatized_tokens)
    })

processed = df[TEXT_COLUMN].apply(preprocess_text)
df = pd.concat([df, processed], axis=1)

df["sentiment"] = df["rating"].apply(label_sentiment)

df = df.drop_duplicates(subset=["lemmatized_text"])
df = df[df["lemmatized_text"].str.len() > 5]

df.to_csv("data/foodpanda_reviews_preprocessed.csv", index=False, encoding="utf-8-sig")

print("Preprocessing completed.")
print("Total records after preprocessing:", len(df))
print(df["sentiment"].value_counts())
print(df[["review_text", "cleaned_text", "lemmatized_text", "sentiment"]].head())
```

Appendix C: Foodpanda Reviews Data Preprocessing Code